LIGHT SENSING & AUTO LIGHTING

Curiosity Kickoff

- What sensor is used to detect light?
- Can system take decisions automatically?
- What is logic in a system?

Concept Connection

A Smart Sensor System uses sensors like LDR and sound detectors to control LEDs automatically. It turns lights ON in darkness,reacts to sound, and uses logic circuits to control.

1.Watt SMD LED

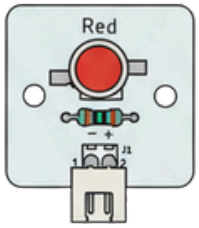
I'm flat and powerful. Just one of me can light up your whole table like a torch.I don't like wasting energy, so I shine super bright without getting too hot,You've already met me — I live in your mobile flashlight!

2,LDR Sensor :

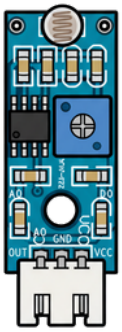
A light sensor whose resistance changes depending on light intensity. Lower resistance in bright light, higher in darkness.

Pinout: 4 Pins - VCC (+), Ground (-), Digital Pin (D0) & Analog Pin (A0)

1



2



Beginner: Light Sense

Class Explanation

In this class, students learn An LDR (Light Dependent Resistor) is a sensor whose resistance changes based on the amount of light falling on it. In bright light, its resistance decreases, and in darkness, its resistance increases.
By connecting an LDR to a circuit with an LED, we can automatically control the LED based on light intensity. For example, the LED can turn ON in the dark and OFF in bright light, making it useful for applications like automatic street lights and light-sensitive switches.

Materials Required

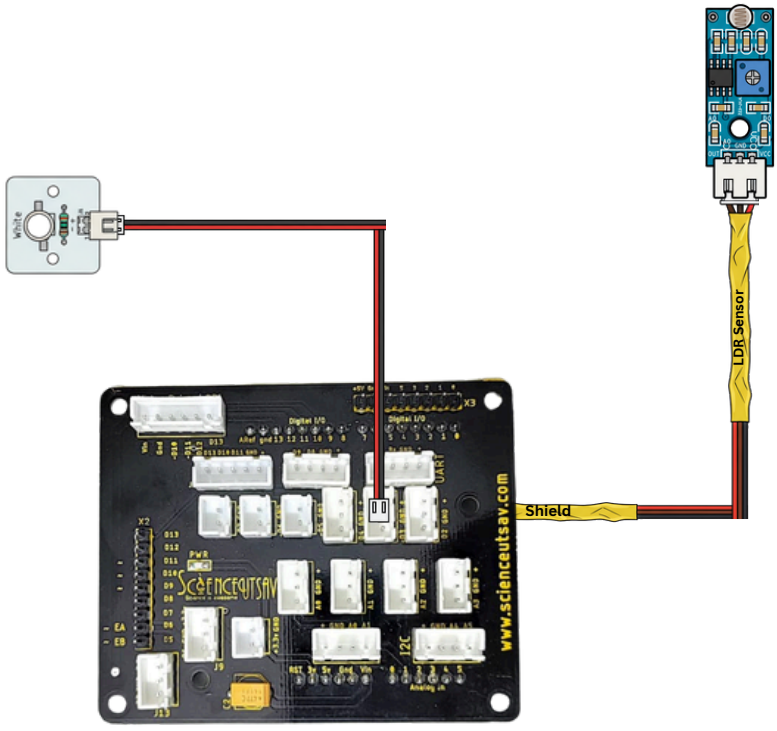
- Arduino with attached shield
- 1 × LDR
- 1 × white 1 Watt SMD LED on breakout board
- RMC wires
- USB cable

How to Connect

Use RMC wires to connect :

- 1 × LDR→ Arduino pin D2 and GND
- 1 × white 1 Watt SMD LED → Arduino pin D3 and GND

Then connect the Arduino to your computer with the USB cable.
“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define LDR_PIN 2  // LDR sensor connected to D2
#define LED_PIN 3  // LED connected to D3

void setup() {
 pinMode(LDR_PIN, INPUT);
 pinMode(LED_PIN, OUTPUT);
 digitalWrite(LED_PIN, LOW);
}
void loop() {
 int ldrstate = digitalRead(LDR_PIN);

 if (ldrstate == HIGH) {
  digitalWrite(LED_PIN, HIGH); // Turn LED ON
 } else {
  digitalWrite(LED_PIN, LOW); // Turn LED OFF
 }
}
```

Code Explanation

- In this Arduino program, we first define the pins using #define. The LDR sensor is connected to pin 2 (LDR_PIN) and the LED is connected to pin 3 (LED_PIN). This makes the code easier to read and use.

- Next, the setup() function runs only once when the Arduino starts. Inside this, we use pinMode() to set the LDR pin as INPUT and the LED pin as OUTPUT. We also use digitalWrite() to make sure the LED is initially turned OFF.

- After that, the loop() function runs continuously. Inside the loop, we read the LDR sensor value using digitalRead() and store it in a variable called ldrstate.

- Then, we use an if-else condition to control the LED. If the LDR detects light (HIGH), the LED turns ON. Otherwise, the LED turns OFF.

So overall, the Arduino continuously checks the light level using the LDR and controls the LED based on the light intensity.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Light Sense Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define LDR_PIN 2   // LDR sensor connected to D2
#define LED1_PIN 3  // First LED connected to D3
#define LED2_PIN 4  // Second LED connected to D4

void setup() {
  pinMode(LDR_PIN, INPUT);
  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);

  digitalWrite(LED1_PIN, LOW);
  digitalWrite(LED2_PIN, LOW);
}

void loop() {
  int ldrstate = digitalRead(LDR_PIN);

  if (ldrstate == HIGH) {
    digitalWrite(LED1_PIN, HIGH); // LED1 ON
    digitalWrite(LED2_PIN, HIGH); // LED2 ON
  } else {
    digitalWrite(LED1_PIN, LOW);  // LED1 OFF
    digitalWrite(LED2_PIN, LOW);  // LED2 OFF
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Light Sense Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Intermediate: Smart Night Light System

Class Explanation

In this class, students learn An LDR (Light Dependent Resistor) is a sensor that detects light intensity. It is used in circuits to control devices like LEDs automatically.
When there is darkness, the LDR's resistance increases, allowing current to flow in a way that turns the LED ON. When there is light, its resistance decreases, causing the LED to turn OFF. Thus, an LDR helps in automatically switching an LED ON in the dark and OFF in the presence of light.

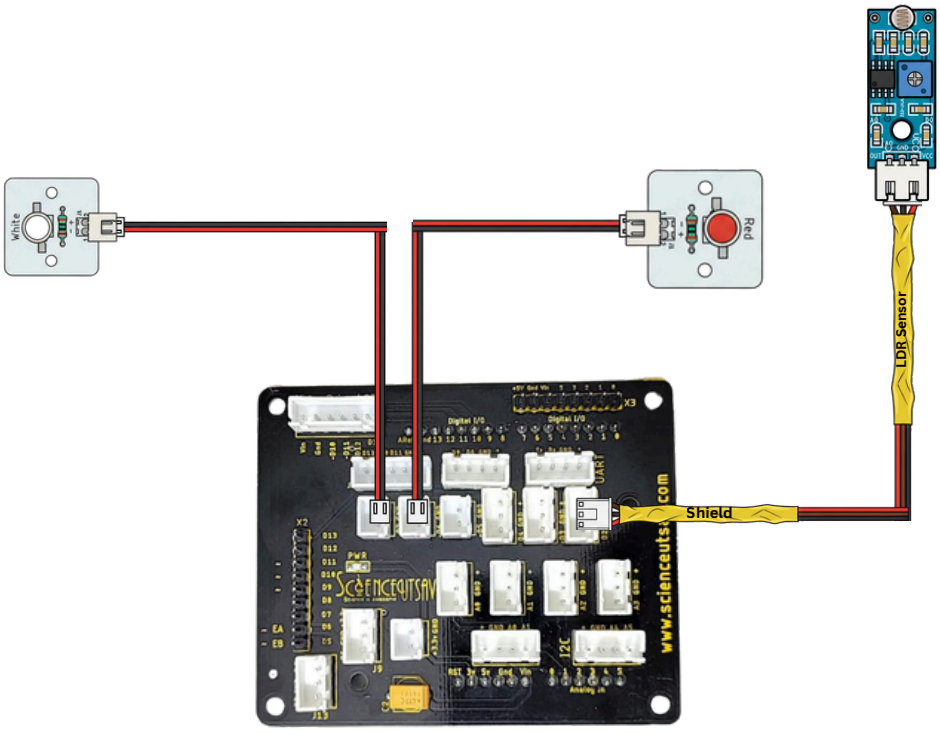
Materials Required

- Arduino with attached shield
- 1 × LDR
- 1 × white 1 Watt SMD LED on breakout board
- 1 × Red 1 Watt SMD LED on breakout board
- RMC wires
- USB cable

How to Connect

- Use RMC wires to connect :
- 1 × LDR→ Arduino pin D2 and GND
- 1 × white 1 Watt SMD LED → Arduino pin D6 and GND
- 1 × RED 1 Watt SMD LED→ Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.
"Refer to the link or QR at the back of the book to get the code file for this project."



Code

```
#define LDR_PIN 2 // LDR sensor connected to D2
#define LED_PIN1 6 // LED connected to D6
#define LED_PIN2 7 // LED connected to D7 (works opposite to D6)

void setup() {
 pinMode(LDR_PIN, INPUT);
 pinMode(LED_PIN1, OUTPUT);
 pinMode(LED_PIN2, OUTPUT);
 digitalWrite(LED_PIN1, LOW);
 digitalWrite(LED_PIN2, LOW);
}

void loop() {
 int ldrstate = digitalRead(LDR_PIN);

 if (ldrstate == HIGH) {
  digitalWrite(LED_PIN1, HIGH); // Turn LED at D6 ON
  digitalWrite(LED_PIN2, LOW); // Turn LED at D7 OFF (opposite behavior)
 } else {
  digitalWrite(LED_PIN1, LOW); // Turn LED at D6 OFF
  digitalWrite(LED_PIN2, HIGH); // Turn LED at D7 ON (opposite behavior)
 }
}
```

Code Explanation

- In this Arduino program, we first define the pins using #define. The LDR sensor is connected to pin 2 (LDR_PIN), and two LEDs are connected to pins 6 (LED_PIN1) and 7 (LED_PIN2). The second LED works in the opposite way to the first LED.

- Next, the setup() function runs only once when the Arduino starts. Inside setup, we use pinMode() to set the LDR pin as INPUT and both LED pins as OUTPUT. We also use digitalWrite() to turn both LEDs OFF initially.

- After that, the loop() function runs continuously. Inside the loop, we read the LDR sensor value using digitalRead() and store it in a variable called ldrstate.

- Then, we use an if-else condition to control both LEDs. If the LDR detects light (HIGH), the LED on pin 6 turns ON and the LED on pin 7 turns OFF. If there is no light (LOW), the LED on pin 6 turns OFF and the LED on pin 7 turns ON.

So overall, the Arduino keeps checking the light level and controls the two LEDs in opposite ways based on the light intensity.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Smart Night Light System with Arduino but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define LDR_PIN 2     // LDR sensor connected to D2
#define LED_PIN 6     // Only one LED on D6
#define BUZZER_PIN 5  // Buzzer on D5

void setup() {
  pinMode(LDR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
}

void loop() {
  int ldrstate = digitalRead(LDR_PIN);

  if (ldrstate == HIGH) {
    digitalWrite(LED_PIN, HIGH);    // LED ON
    digitalWrite(BUZZER_PIN, HIGH); // Buzzer ON
  } else {
    digitalWrite(LED_PIN, LOW);     // LED OFF
    digitalWrite(BUZZER_PIN, LOW);  // Buzzer OFF
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:
the basic idea of Smart Night Light System with Arduino how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Advanced: Smart Light Detection

Class Explanation

In this class, students learn the A Smart Light Detection system uses sensors like an LDR to detect light intensity and control devices automatically. It turns lights ON in darkness and OFF in bright conditions, improving energy efficiency and enabling automatic lighting in homes, streets, and smart electronic systems.

Materials Required

- Arduino with attached shield
- 1 × LDR
- 2 × Buzzer
- RMC wires
- USB cable

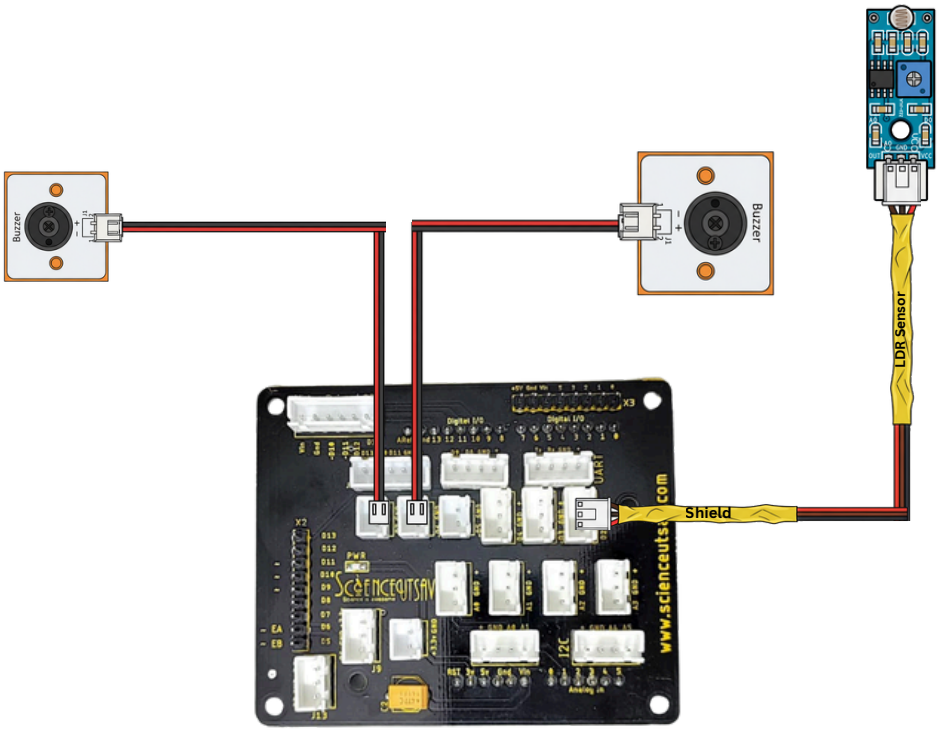
How to Connect

Use RMC wires to connect :

- 1 × LDR sensor→ Arduino pin D2 and GND
- 1 × Buzzer → Arduino pin D6 and GND
- 1 × Buzzer → Arduino pin D7 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define LDR_PIN 2     // LDR sensor on D2
#define BUZZ1_PIN 6   // Buzzer 1 on D6
#define BUZZ2_PIN 7   // Buzzer 2 on D7 (opposite)

void setup() {
  pinMode(LDR_PIN, INPUT);
  pinMode(BUZZ1_PIN, OUTPUT);
  pinMode(BUZZ2_PIN, OUTPUT);

  digitalWrite(BUZZ1_PIN, LOW);
  digitalWrite(BUZZ2_PIN, LOW);
}

void loop() {
  int ldrstate = digitalRead(LDR_PIN);

  if (ldrstate == HIGH) {
    digitalWrite(BUZZ1_PIN, HIGH); // Buzzer1 ON
    digitalWrite(BUZZ2_PIN, LOW);  // Buzzer2 OFF
  } else {
    digitalWrite(BUZZ1_PIN, LOW);  // Buzzer1 OFF
    digitalWrite(BUZZ2_PIN, HIGH); // Buzzer2 ON
  }
}
```

Code Explanation

- In this Arduino program, an LDR sensor is connected to pin 2 (LDR_PIN), and two LEDs are connected to pins 6 (LED_PIN1) and 7 (LED_PIN2). The second LED works opposite to the first LED.

- The setup() function runs once when the Arduino starts. It sets the LDR as INPUT and both LEDs as OUTPUT using pinMode(). Initially, both LEDs are turned OFF.

- The loop() function runs continuously. It reads the LDR value using digitalRead() and stores it in ldrstate.

- If light is detected (HIGH), the LED on pin 6 turns ON and the LED on pin 7 turns OFF. If there is no light (LOW), the LED on pin 6 turns OFF and the LED on pin 7 turns ON.

So overall, the Arduino controls two LEDs in opposite ways based on light intensity.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Smart Light Detection but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define LDR_PIN 2     // LDR sensor on D2
#define BUZZER_PIN 6  // Buzzer on D6

void setup() {
  pinMode(LDR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);
}

void loop() {
  int ldrstate = digitalRead(LDR_PIN);

  if (ldrstate == HIGH) {
    digitalWrite(BUZZER_PIN, HIGH); // Bright → Buzzer ON
  } else {
    digitalWrite(BUZZER_PIN, LOW);  // Dark → Buzzer OFF
  }
}
```

Obective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:
the basic idea of Smart Light Detection how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Expert: Adaptive Light Logic

Class Explanation

In this class, students learn the basic idea of An Adaptive Light Logic system uses light sensors like an LDR combined with logic control to adjust lighting based on surrounding conditions. It automatically adapts brightness or switches lights ON/OFF depending on light intensity, improving efficiency, comfort, and smart automation in electronic systems.

Materials Required

- Arduino with attached shield
- 1 × LDR sensor
- 2 × white 1 Watt SMD LED on breakout board
- 1 × Red 1 Watt SMD LED on breakout board
- 1 × Buzzer
- RMC wires
- USB cable

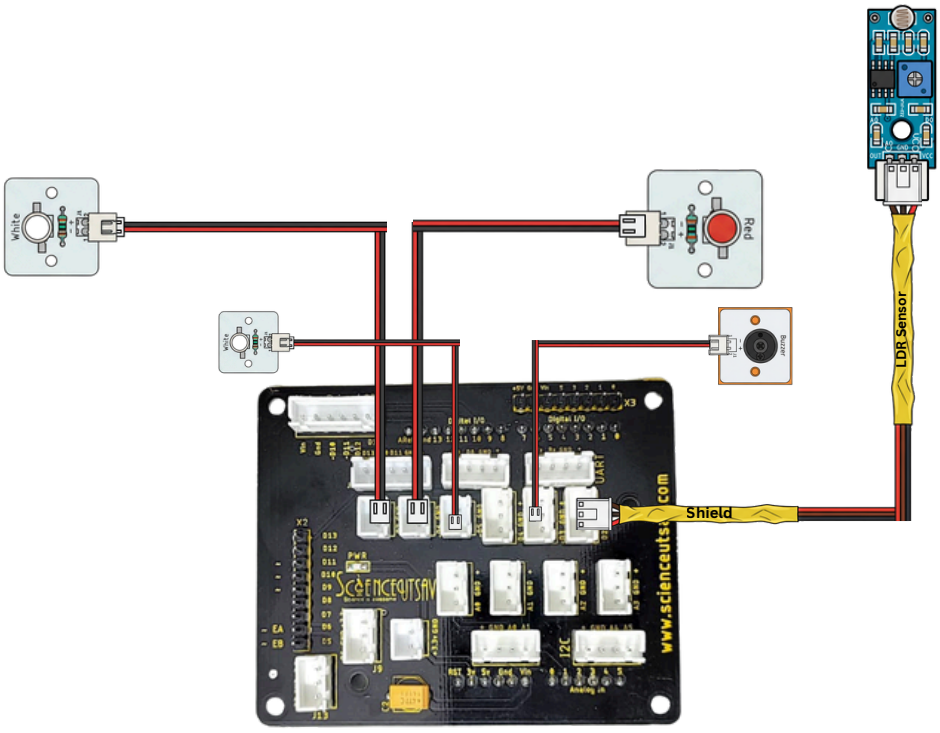
How to Connect

Use RMC wires to connect :

- 1 × LDR sensor → Arduino pin D2 and GND
- 1 × white 1 Watt SMD LED → Arduino pin D5 and GND
- 1 × Red 1 Watt SMD LED → Arduino pin D6 and GND
- 1 × white 1 Watt SMD LED → Arduino pin D7 and GND
- 1 × Buzzer → Arduino pin D3 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define LDR_PIN 2      // LDR sensor on D2

#define LED1_PIN 5     // LED1 on D5
#define LED2_PIN 6     // LED2 on D6
#define LED3_PIN 7     // LED3 on D7

#define BUZZER_PIN 3   // Buzzer on D3

void setup() {
  pinMode(LDR_PIN, INPUT);

  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);
  pinMode(LED3_PIN, OUTPUT);

  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(LED1_PIN, LOW);
  digitalWrite(LED2_PIN, LOW);
  digitalWrite(LED3_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
}

void loop() {
  int ldrState = digitalRead(LDR_PIN);

  // Most LDR modules: LOW = dark, HIGH = bright
  if (ldrState == LOW) {
    // Dark → turn everything ON
    digitalWrite(LED1_PIN, HIGH);
    digitalWrite(LED2_PIN, HIGH);
    digitalWrite(LED3_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
  } else {
    // Bright → turn everything OFF
    digitalWrite(LED1_PIN, LOW);
    digitalWrite(LED2_PIN, LOW);
    digitalWrite(LED3_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
  }
}
```

Code

```
#define LDR_PIN 2      // LDR sensor on D2

#define LED1_PIN 3     // LED1 on D3
#define LED2_PIN 4     // LED2 on D4
#define LED3_PIN 5     // LED3 on D5

#define BUZZER_PIN 6   // Buzzer on D6

void setup() {
  pinMode(LDR_PIN, INPUT);

  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);
  pinMode(LED3_PIN, OUTPUT);

  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(LED1_PIN, LOW);
  digitalWrite(LED2_PIN, LOW);
  digitalWrite(LED3_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
}

void loop() {
  int ldrState = digitalRead(LDR_PIN);

  // Most LDR modules: LOW = dark, HIGH = bright
  if (ldrState == LOW) {
    // Dark → turn everything ON
    digitalWrite(LED1_PIN, HIGH);
    digitalWrite(LED2_PIN, HIGH);
    digitalWrite(LED3_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
  } else {
    // Bright → turn everything OFF
    digitalWrite(LED1_PIN, LOW);
    digitalWrite(LED2_PIN, LOW);
    digitalWrite(LED3_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
  }
}
```

Code Explanation

- In this Arduino program, an LDR sensor is connected to pin 2 (LDR_PIN), three LEDs are connected to pins 3, 4, and 5 (LED1_PIN, LED2_PIN, LED3_PIN), and a buzzer is connected to pin 6 (BUZZER_PIN).

- The setup() function runs once when the Arduino starts. It sets the LDR as INPUT and all LEDs and the buzzer as OUTPUT using pinMode(). Initially, all LEDs and the buzzer are turned OFF.

- The loop() function runs continuously. It reads the LDR value using digitalRead() and stores it in ldrState.

- If it is dark (LOW signal), all three LEDs and the buzzer turn ON. If it is bright (HIGH signal), all LEDs and the buzzer turn OFF.

So overall, the system detects light intensity and activates multiple LEDs and a buzzer in darkness.

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:
the basic idea of Adaptive Light Logic how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes
After completing this session, students should be able to:

- Understand how light sensors detect brightness levels and provide input signals for controlling electronic devices automatically.
- Learn how automatic lighting systems turn ON in darkness and OFF in light using sensors for energy-efficient operation.
- Understand how light intensity is measured and used to control outputs like LEDs in smart electronic applications.
- Learn how light sensor inputs are processed using logic to adjust lighting conditions intelligently based on the environment.

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself code
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output